JAVA PRACTICAL PROGRAM REPORT

Bank Account Management Using a Class

Program/Class Name: BankAccountManagement.java

Language: Java

Topic: Class, object, constructor, instance fields, this keyword, methods, deposit operation, withdrawal balance check, and Scanner input.

This report includes the aim, requirements, algorithm, UML-style diagram, complete source code, execution commands, sample console output, and conclusion.

1. Aim

To demonstrate class and object concepts by creating a bank account, displaying account details, depositing money, and attempting a withdrawal.

2. Requirements

• Java Development Kit (JDK)
• Text editor or Java IDE
• Save the file using the public class name.

3. Concepts Used

Class, object, constructor, instance fields, this keyword, methods, deposit operation, withdrawal balance check, and Scanner input.

4. Algorithm

1. Start and create a Scanner object.

2. Create a BankAccount object with account number 101, holder Anu, and initial balance 5000.

3. Display the account number, holder name, and balance.

4. Read a deposit amount and call deposit() to update the balance.

5. Read a withdrawal amount and call withdraw().

6. If withdrawal amount is no greater than the balance, subtract it; otherwise display Insufficient Balance.

7. Close Scanner and end.

5. UML / Class Diagram

BankAccount: accountNumber, accountHolder, balance
Constructor: BankAccount(number, holder, balance)
Methods: displayAccount(), deposit(amount), withdraw(amount)
BankAccountManagement.main(): creates object and reads transactions

The diagram summarizes the main class, its key fields or methods, and the program flow.

6. Source Code

```
import java.util.Scanner;

class BankAccount {

    int accountNumber;
    String accountHolder;
    double balance;

    BankAccount(int accountNumber, String accountHolder, double balance) {
        this.accountNumber = accountNumber;
        this.accountHolder = accountHolder;
        this.balance = balance;
    }

    void displayAccount() {
        System.out.println("Account Number: " + accountNumber);
        System.out.println("Account Holder: " + accountHolder);
        System.out.println("Balance: " + balance);
    }

    void deposit(double amount) {
        balance = balance + amount;
        System.out.println("Amount Deposited Successfully");
        System.out.println("New Balance: " + balance);
    }

    void withdraw(double amount) {
        if (amount <= balance) {
            balance = balance - amount;
            System.out.println("Amount Withdrawn Successfully");
            System.out.println("New Balance: " + balance);
        } else {
            System.out.println("Insufficient Balance");
        }
    }
}

public class BankAccountManagement {
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        BankAccount account =
            new BankAccount(101, "Anu", 5000);

        account.displayAccount();

        System.out.print("Enter deposit amount: ");
        double depositAmount = sc.nextDouble();
        account.deposit(depositAmount);

        System.out.print("Enter withdrawal amount: ");
        double withdrawAmount = sc.nextDouble();
        account.withdraw(withdrawAmount);

        sc.close();
    }
}
```

7. Compilation and Execution

```
javac BankAccountManagement.java
java BankAccountManagement
```

Sample scenario: Deposit 1000; withdraw 2000.

8. Console Output

The code was compiled and run in the available Java environment. Interactive inputs are shown beside prompts for readability.

```
Account Number: 101
Account Holder: Anu
Balance: 5000.0
Enter deposit amount: Amount Deposited Successfully
New Balance: 6000.0
Enter withdrawal amount: Amount Withdrawn Successfully
New Balance: 4000.0
```

9. Result / Conclusion

The program demonstrates encapsulation of account data and behavior in a BankAccount object. Starting from 5000, depositing 1000 gives 6000; withdrawing 2000 leaves a balance of 4000.